

Tutorial de controle de versões pelo git

Passo 1: Navegar até a pasta do seu projeto

Primeiro, você precisa entrar na pasta onde estão os arquivos do seu CRUD. No PowerShell, use o comando `cd` (Change Directory):

PowerShell

```
cd "C:\Caminho\Para\Seu\meu_projeto_crud"
```

Passo 2: Inicializar o Git localmente

Agora, transforme essa pasta comum em um repositório Git local:

PowerShell

```
git init
```

Nota: Isso criará uma pasta oculta chamada `.git` dentro do seu projeto. É ela que vai rastrear suas alterações.

Passo 3: Definir o nome da branch principal

Por padrão antigo, o Git pode criar uma branch chamada `master`, mas o padrão atual do GitHub é `main`. Ajuste isso com o comando:

PowerShell

```
git branch -M main
```

Passo 4: Adicionar os arquivos para o "Stage"

Você precisa dizer ao Git quais arquivos devem ser salvos nesse primeiro envio. Para adicionar todos os arquivos da pasta, use:

PowerShell

```
git add .
```

Passo 5: Fazer o primeiro Commit

O commit é como salvar um "ponto de restauração" do seu código. Adicione uma mensagem explicando o que é esse commit:

PowerShell

```
git commit -m "First commit: Estrutura inicial do CRUD"
```

```
PS C:\Users\alexa\OneDrive\Desktop\Projeto\Pessoal\UTF\Gestão de qualidade\Trabalho controle de versões> git commit -m "Primeiro commit: Estrutura inicial do CRUD"
[main (root-commit) ac31766] Primeiro commit: Estrutura inicial do CRUD
6 files changed, 275 insertions(+)
create mode 100644 README.md
create mode 100644 main.py
create mode 100644 src/_init_.py
create mode 100644 src/crud.py
create mode 100644 src/pessoa.py
create mode 100644 tests/_init_.py
```

Passo 6: Conectar ao GitHub

Agora precisamos dizer ao Git local para onde ele deve enviar esses arquivos na internet.

1. Vá até o seu **GitHub** no navegador.
2. Clique no botão **"New"** (ou no ícone de + no topo direito) para criar um novo repositório.
3. Dê um nome ao repositório (ex: meu_projeto_crud) e clique em **"Create repository"** (deixe as opções de README e .gitignore desmarcadas, pois já estamos criando isso localmente).
4. Na página que abrir, copie a URL do repositório (ela termina com .git).

Volte ao PowerShell e digite o comando abaixo, substituindo pela URL que você copiou:

PowerShell

```
git remote add origin https://github.com/seu-usuario/seu-repositorio.git
```

Passo 7: Enviar o código (Push)

Por fim, envie os seus arquivos locais para o servidor do GitHub:

PowerShell

```
git push -u origin main
```

CASO TENHA ALGUM CONFLITO:

O que aconteceu aqui foi o seguinte: quando você criou o repositório no GitHub, você provavelmente marcou a opção de criar um arquivo **README.md** ou um **.gitignore** por lá. O GitHub criou esses arquivos e gerou um "histórico" inicial. Como você também criou um histórico na sua máquina (com o git init e git commit), o Git olhou para os dois e disse: *"Ei, são dois projetos totalmente diferentes, não posso juntar eles porque as histórias não batem"* (daí o erro fatal: refusing to merge unrelated histories).

Passo 1: Permitir a junção de históricos diferentes

Precisamos forçar o Git a aceitar puxar os arquivos do GitHub (o README que está lá) para dentro do seu projeto local, mesmo que eles tenham históricos diferentes.

Execute exatamente este comando:

PowerShell

```
git pull origin main --allow-unrelated-histories
```

Passo 2: Resolver o conflito (se houver) ou salvar a mensagem

Ao rodar o comando acima, podem acontecer duas coisas:

1. **Tudo dar certo diretamente:** O Git vai baixar o arquivo do GitHub e juntar com os seus. Se ele abrir uma tela de texto estranha (geralmente o editor Vim), calma! É só ele pedindo uma mensagem de commit. Para sair dela: digite `:wq` e aperte Enter.
2. **Dar conflito de arquivos:** Se você já tinha um arquivo com o mesmo nome (como um README.md) no seu computador e no GitHub, o Git vai avisar que há um conflito. Se isso acontecer, abra o arquivo no seu VS Code, escolha qual versão manter e faça um novo commit:

PowerShell

```
git add .  
git commit -m "Fix: juntando historicos"
```

Passo 3: Enviar o código atualizado para o GitHub

Agora que o seu computador tem o histórico sincronizado com o do GitHub, o comando de envio vai funcionar perfeitamente. Digite:

PowerShell

```
git push -u origin main
```

Criação de branches:

No Git, criar branches é a melhor prática para desenvolver novas funcionalidades ou fazer correções sem mexer no código principal (main) que está funcionando.

Para criar e já entrar na branch **correções**, execute o seguinte comando no seu PowerShell:

PowerShell

```
git checkout -b correcoes
```

Dica de boa prática: Evite usar caracteres especiais, espaços ou acentos em nomes de branches. Por isso, usamos `correcoes` (sem o `ç` e o `õ`).

O que esse comando faz por baixo dos panos?

Ele é a união de duas ações:

1. `git branch correcoes`: Cria a branch com esse nome.
2. `git checkout correcoes`: Muda o seu terminal da branch main para a branch `correcoes`.

Para confirmar que você mudou de branch com sucesso, você pode digitar:

PowerShell

```
git branch
```

O terminal vai listar as branches existentes e colocar um asterisco * verde na branch em que você está posicionado no momento.

E como enviar essa nova branch para o GitHub?

Depois que você fizer as alterações no seu CRUD, salvar com `git add .` e `git commit`, o primeiro envio dessa nova branch para o GitHub precisa de um comando levemente diferente:

PowerShell

```
git push -u origin correcoes
```

A partir do segundo envio nessa mesma branch, você poderá usar apenas `git push`.

Rastreio de versões

No contexto de desenvolvimento de software e do Git, o termo **baseline** representa a sua "linha de base", ou seja, uma versão estável, homologada e conhecida do código (geralmente a sua branch principal `main` ou `master`, ou uma versão marcada com uma `tag`).

Quando você está trabalhando em uma branch de correções (`correcoes`), o Git oferece excelentes ferramentas direto no terminal do VS Code para você rastrear o histórico e comparar o seu progresso atual com essa baseline.

Aqui estão os principais comandos para fazer isso:

1. Ver o histórico completo e a árvore de commits (A Baseline Visual)

Para ver como a sua branch atual se comporta em relação à baseline (`main`), o comando abaixo desenha uma árvore do histórico direto no terminal:

PowerShell

```
git log --oneline --graph --all
```

- **O que ele mostra:** Ele exibirá uma linha do tempo compacta. Você conseguirá ver exatamente o ponto onde a sua branch `correcoes` se separou da `main` (a sua linha de base).

```
* 1d255b4 (HEAD -> opcao_deletar, origin/main, origin/HEAD, main, correcoes) versão 3.0 implementada atualização
* 7537ac9 versão 2.0 com buscas por ID,cpf e nome
* b625a0c Merge branch 'main' of https://github.com/alexoliao/CRUD-em-python
| \
| * 0524fca Initial commit
* ac31766 Primeiro commit: Estrutura inicial do CRUD
```

2. Comparar seu código atual com a Baseline (Ver as diferenças)

Se você quer ver quais linhas de código você alterou na sua branch de correções em relação à branch principal, use o `git diff`:

PowerShell

```
git diff main
```

- **O que ele mostra:** O terminal vai listar os arquivos modificados. O que estiver em **vermelho** (com o sinal -) é o que está na baseline mas foi alterado/removido por você. O que estiver em **verde** (com o sinal +) são as suas correções atuais.

Se quiser ver apenas o nome dos arquivos que mudaram, sem o código interno:

PowerShell

git diff main --name-status

```
PS C:\Usegit diff mainrive\Desktop\Projeto\Pessoal\UTF\Gestão de qualidade\Trabalho controle de versões>
diff --git a/main.py b/main.py
index 2c9e0af..ad67459 100644
--- a/main.py
+++ b/main.py
@@ -219,14 +219,49 @@ def atualizar_pessoa(crud):
     else:
         print(f"✖ Erro ao atualizar pessoa.")

-     except ValueError:
-         print("✖ Erro: ID deve ser um número inteiro!")
-     except ValueError as e:
-         ...skipping...
diff --git a/main.py b/main.py
index 2c9e0af..ad67459 100644
--- a/main.py
+++ b/main.py
@@ -219,14 +219,49 @@ def atualizar_pessoa(crud):
     else:
         print(f"✖ Erro ao atualizar pessoa.")

-     except ValueError:
-         print("✖ Erro: ID deve ser um número inteiro!")
-     except ValueError as e:
-         print(f"✖ {e}")
-     except Exception as e:
-         print(f"✖ Erro ao atualizar: {e}")

+def deletar_pessoa(crud):
+    """Menu para deletar uma pessoa."""
+    print("\n" + "=" * 50)
+    print("DELETAR PESSOA")
+    print("=" * 50)
+    try:
+        # Busca a pessoa por ID
+        id_str = input("Digite o ID da pessoa a deletar: ").strip()
+        id = int(id_str)
```

3. Verificar o rastreamento (Onde sua branch está apontando)

Para ter certeza de que a sua branch local está rastreando a versão correta do servidor (GitHub), use o comando de listagem detalhada:

PowerShell

git branch -vv

- **O que ele mostra:** Ele listará suas branches locais e mostrará, entre colchetes [...], qual é a branch correspondente no GitHub (ex: [origin/correcoes]). Isso garante que você está rastreando o ponto certo da sua linha de desenvolvimento.

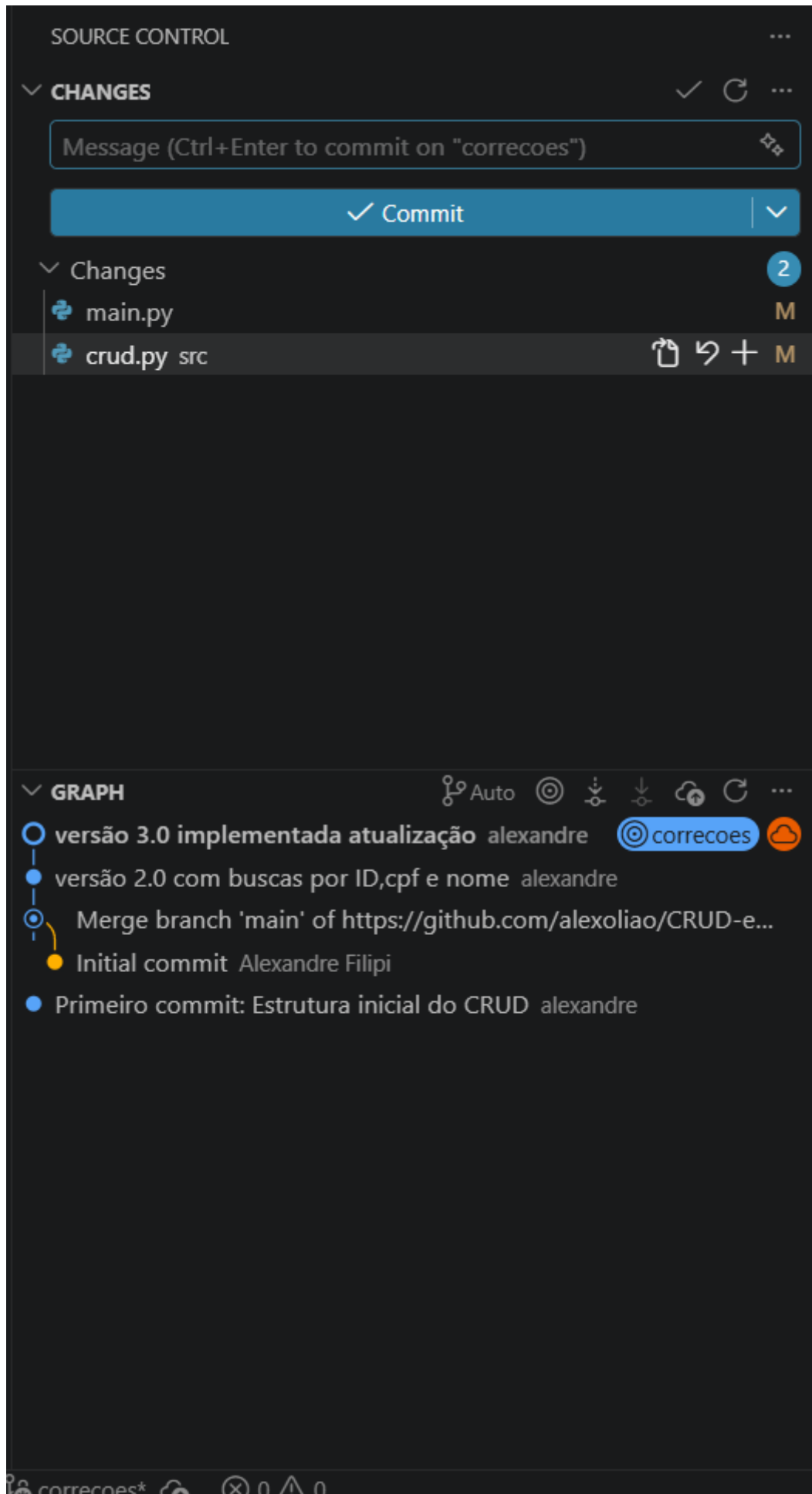
```
PS C:\Users\alexa\OneDrive\Desktop\Projeto\Pessoal\UTF\Gestão de qualidade\Trabalho controle de versões> git branch -vv
correcoes      1d255b4 versão 3.0 implementada atualização
main           1d255b4 [origin/main] versão 3.0 implementada atualização
* opcao_deletar 1d255b4 versão 3.0 implementada atualização
```

Dica extra no VS Code (Interface Visual)

Embora você tenha perguntado pelo terminal, o VS Code tem um visualizador de *baseline* nativo espetacular:

1. No menu lateral esquerdo, clique no ícone de **Source Control** (o símbolo do Git com uma ramificação).
2. Na barra amarela/azul no rodapé do VS Code, você consegue ver em qual branch está posicionado.

3. Se você instalar a extensão chamada **GitLens**, ela cria uma aba chamada



"Commits" onde você consegue ver graficamente a sua linha de base e a distância da sua branch atual em relação a ela.